Last Resort Hotels

Group 9
Team Members: Saket Bansal, Jasmine Ji, Ibne Tamim & Andy Yan

Team Organization and Shared Understanding

For our Last Resort Hotels project, we understand that working as a team means more than simply assigning everyone a different task. The ERD, normalization process, and written report are all connected, so we need to make sure our individual contributions fit together and reflect the same database design. To keep things organized, we decided to divide the work based on different parts of the project, giving each member a clear responsibility while still encouraging everyone to share ideas, review each other's work, and help make decisions as a group.

Division of Responsibilities

Saket Bansal- ERD

Jasmine Ji- Database Normalization and Validation

Andy Yan- ERD

Ibne Tamim- Draft Analysis Report

Accountability and Communication

To stay organized, we plan to use our group chat to keep each other updated, ask questions, and discuss any issues that come up while working on the project. We want to finish our individual tasks ahead of the deadline so that everyone has enough time to review the work and make changes if needed.

Each member is responsible for keeping the group informed about their progress, especially if they run into difficulties or need more time. If someone is struggling with a particular task, we will work together to find a solution or adjust the responsibilities so that no one is left handling everything on their own. Even though we have divided the work, we still want everyone to be familiar with the entire project. Before submitting, each of us will review the ERD, analysis report, and team statement to make sure everything is consistent and accurately represents the decisions we made as a group. We believe that staying involved in each other's work will help us catch mistakes early and produce a stronger final submission.

Ensuring an Equitable Workload

Although our responsibilities differ in nature, we intend to distribute the overall workload fairly. The ERD and normalization review require substantial technical attention, while the analysis report and team statement require careful documentation, coordination, and consistency checks. We will evaluate the

distribution of work throughout the project rather than assuming that an initial assignment guarantees an equal contribution. If one component proves more demanding than anticipated, other members will assist with revisions, validation, documentation, or repository management. This approach is intended to prevent any one member from carrying a disproportionate share of the project.

Collaboration and Final Review

Before submitting Milestone 1, our group will review the ERD, analysis report, and team statement together to ensure that the three deliverables present a consistent understanding of the proposed database. In particular, we will verify that the entity names, attributes, relationships, and assumptions described in the report match those shown in the ERD. We view this milestone not simply as a collection of separate assignments, but as the foundation for the database we will continue developing throughout the semester. Our goal is to establish a working process in which responsibilities are clear, questions can be raised openly, and each member remains accountable for the quality of the project as a whole

---

1. Introduction and Project Motivation Managing a hotel involves much more than keeping track of available rooms and reservations. Each room belongs to a specific floor, wing, building, and hotel property. Hotels also need to keep track of customer information, room details, and booking dates. While managing this information might be relatively simple for a small hotel, it becomes more challenging when a company operates multiple properties with different layouts and room configurations.

For our project, *Last Resort Hotels*, we are designing a database for a hypothetical hotel company that manages several properties. Each hotel may have multiple buildings, which are divided into wings, floors, and individual rooms. Along with keeping track of these locations, the company also needs a way to organize customer records and connect reservations to the correct rooms.

One of the main challenges we considered was how to organize all this information without repeating the same details across different records. For example, every room should have a clearly defined location, and every reservation should be connected to both a customer and a room. At the same time, if a hotel changes its address, we should be able to update that information in one place rather than having to change it in every room or reservation record.

With these considerations in mind, we decided to design a relational database that separates information into different but connected tables. For this first milestone, our focus is on identifying the main entities, defining how they relate to one another, explaining the assumptions behind our decisions, and making sure our design follows the principles of Third Normal Form (3NF).

Our goal is to create a database that reflects how a hotel operates in practice while keeping the structure organized, manageable, and flexible enough to build upon as the project progresses.

2. Speculative Organizational Scenario

We envision Last Resort Hotels as a growing hospitality company operating multiple properties with different physical layouts and room configurations.

One property might consist of a single building, while another could contain several buildings divided into separate wings. Individual floors contain rooms that differ in size, bed configuration, smoking policy, and guest capacity.

The company needs a reliable way to answer questions that arise during ordinary hotel operations. For instance:

- Which rooms belong to a particular building or wing?
- What are the characteristics and guest capacities of those rooms?
- Which customer made a particular reservation?
- What room was reserved, and for what period?
- What rate was recorded when the reservation was made?

Although these questions appear relatively simple, answering them consistently requires several kinds of information to be connected.

Consider a customer reserving a room at one of the company's properties. The customer provides identifying and contact information, selects a room, and establishes a check-in and check-out period. The reservation must preserve these details while maintaining a link to the room's physical location.

If the customer returns several months later, the company should not need to create an entirely new customer record. Similarly, if the hotel changes its address, that change should be reflected in the hotel's record rather than requiring modifications to every reservation associated with the property.

Our proposed schema is designed around these relationships. We deliberately limit the initial scope to hotel organization, room inventory, customer information, and reservations. Although payment processing, housekeeping, maintenance, and staffing are important aspects of hotel management, incorporating them at this stage would introduce additional relationships beyond those needed to establish the core database. This narrower scope allows us to concentrate on the integrity and organization of the data before expanding the system.

3. Proposed Database Structure

Our preliminary schema consists of seven entities: Hotel, Buildings, Wings, Floor, Room, Customer, and Reservation. The first five represent the physical organization of the hotel properties. The remaining two represent the customers who use those properties and the reservations they make. This distinction is central to our design. The physical characteristics of a hotel exist independently of any particular reservation, while reservations are transactions that refer to those existing records.

3.1 Hotel

The Hotel entity represents an individual property operated by Last Resort Hotels.

Its attributes are:

- hotelID - Primary Key

- name
- address
- city
- country

We use a unique hotel identifier rather than relying on the hotel's name because names are not necessarily unique and may change over time. A hotel can contain multiple buildings, but each building belongs to one hotel. Keeping hotel information in its own table also avoids repeating property names and addresses throughout the database.

3.2 Buildings

The Buildings entity represents the individual buildings belonging to a hotel property.

Its attributes are:

- buildingID - Primary Key
- hotelID - Foreign Key
- name

The hotelID foreign key establishes the relationship between a building and its parent hotel.

We distinguish buildings from hotels because a property may occupy more than one physical structure. Combining these entities would either restrict the model to single-building properties or require repeated hotel information.

Separating them gives the organization flexibility to represent properties of different sizes.

3.3 Wings

The Wings entity represents subdivisions within a building.

Its attributes are:

- wingID - Primary Key
- buildingID - Foreign Key
- wingName

Not every hotel organizes its rooms into wings, but our scenario assumes that every room is assigned to a wing, even if a smaller building contains only one.

This assumption provides a consistent structure across the organization's properties.

The relationship between Buildings and Wings is one-to-many: a building can contain several wings, while each wing belongs to one building.

3.4 Floor

The Floor entity represents a floor within a particular wing.

Its attributes are:

- floorID - Primary Key
- wingID - Foreign Key
- floorNumber

Floor numbers alone are insufficient as unique identifiers because different buildings and wings may contain floors with identical numbers.

We therefore assign each floor its own primary key.

The wingID foreign key allows the organization to identify the wing, building, and hotel associated with a particular floor.

3.5 Room

The Room entity represents an individual guest room.

Our preliminary attributes include:

- roomID - PK
- floorID - FK
- room_number
- number_of_beds
- bed1_size
- bed2_size
- extra_space
- smoking
- amt_guests

The Room entity is particularly important because it connects the hotel's physical organization to its reservation activity.

Each room belongs to exactly one floor. Through the relationships established by the preceding entities, the room can also be associated with a wing, building, and hotel without storing those identifiers redundantly. We distinguish roomID from room_number because different hotels may use the same room numbers. Room 205, for example, could exist in several properties.

The current design includes separate attributes for the sizes of the first and second beds. This reflects an initial assumption that rooms contain no more than two beds. We recognize that this limits flexibility. If the organization needs to accommodate an arbitrary number of beds, introducing a separate Bed entity would be more appropriate.

We will evaluate this decision as part of our normalization review.

3.6 Customer

The Customer entity stores information about individuals or organizations making reservations.

Its attributes are:

- customerID- PK
- customerType
- name
- phone
- billingAddress

A customer may make multiple reservations, but their identifying and contact information should not need to be duplicated with every booking.

We therefore separate customers from reservations and connect the two through a foreign key.

The customerType attribute allows the organization to distinguish between categories of customers, such as individuals and corporate accounts, without creating entirely separate customer tables.

For the initial design, we assume that the listed attributes are sufficient to represent both categories.

3.7 Reservation

The Reservation entity records a customer's booking of a room.

Its proposed attributes are:

- reservationID- PK
- customerID - FK
- roomID - FK
- reservationTime
- checkinTime
- checkoutTime
- actualRate

Unlike the other entities, Reservation represents an event rather than a physical component or relatively stable customer record.

Each reservation is associated with one customer and one room. A customer may make multiple reservations, and a room may appear in multiple reservations over time.

The reservationTime attribute records when the booking was made, while checkinTime and checkoutTime define the intended occupancy period.

We retain actualRate within Reservation because the amount charged for a booking may differ from a room's standard price. Rates can change depending on the season, availability, or promotional arrangements. Recording the actual rate preserves the amount associated with that particular transaction.

Our current design assumes that one reservation corresponds to one room. If the organization later wishes to support a single reservation covering multiple rooms, we would introduce a separate relationship between reservations and rooms.

4. Relationships and Cardinality

The schema uses one-to-many relationships to represent both the hotel's physical hierarchy and its reservation activity.

Parent Entity	Related Entity	Relationship
Hotel	Buildings	One hotel can have many buildings
Buildings	Wings	One building can have many wings
Wings	Floor	One wing can have many floors
Floor	Room	One floor can have many rooms
Customer	Reservation	One customer can make many reservations
Room	Reservation	One room can appear in many reservations over time

These relationships establish how records depend on one another. For example, a room is not directly assigned to a hotel through a separate hotel identifier. Instead, its hotel can be determined by following the relationships from Room to Floor, Wings, Buildings, and finally Hotel. This approach avoids storing the same location information in several places. It also creates a clearer basis for referential integrity. A reservation should not reference a nonexistent room, and a building should not reference a hotel that does not exist in the database. We intend to enforce these relationships through primary-key and foreign-key constraints during implementation.

5. Design Assumptions and Constraints

Because Last Resort Hotels is a speculative organization, certain operational decisions must be established before the database can be designed consistently.

Our principal assumptions are as follows:

Physical organization. Every building belongs to one hotel, every wing belongs to one building, every floor belongs to one wing, and every room belongs to one floor. A property may contain multiple buildings, but these relationships do not overlap across properties.

Unique identification. Each entity uses a unique identifier as its primary key. Names, room numbers, and floor numbers are not assumed to be globally unique.

Customer records. Customers may make multiple reservations, but each reservation is associated with one primary customer. Additional guests accompanying that customer are not represented as separate reservation participants in the current schema.

Room bookings. Each reservation corresponds to one room. A room may be booked repeatedly over time, but overlapping active reservations for the same room should not be permitted.

Reservation dates. A reservation's check-out time must occur after its check-in time. The reservation creation time is recorded separately from the intended stay.

Room capacity. The number of guests occupying a room should not exceed its designated capacity. The initial schema stores room capacity but does not yet include a separate field for the number of guests associated with a particular reservation.

Pricing. The actual reservation rate is stored with the booking. The present design does not attempt to model seasonal pricing rules, discounts, taxes, or payment transactions.

Scope of operations. The database focuses on room inventory and reservations rather than serving as a complete hotel enterprise-management system.

These assumptions help define the boundaries of the project. They also identify areas where the schema could be expanded if additional business requirements were introduced.

6. Normalization and Data Integrity

Normalization is an important part of our database design because hotel information is connected across several levels, from individual rooms to floors, buildings, and hotel properties. Without properly organizing this information, we could end up storing too many details in a single table. For example, if every reservation included the customer's name, phone number, hotel address, building name, floor number, and room details, much of that information would be repeated unnecessarily.

This could become a problem as the hotel grows. If a customer makes five different reservations, their contact information might appear five times. Similarly, a hotel's address could be repeated across hundreds of records. If any of these details changed, we would have to update every record containing that information, increasing the chances of mistakes or inconsistencies.

To avoid these issues, we decided to separate the information into different tables based on what each table represents. This allows us to store information in one place and connect it to other records when needed, making the database easier to manage and update.

First Normal Form (1NF)

First Normal Form requires attributes to contain atomic values and prohibits repeating groups within a relation. Our design assigns a primary key to each entity and generally stores one value per attribute. One area requiring particular attention is the representation of beds within Room.

The attributes bed1_size and bed2_size are separate fields, rather than a single list of bed sizes. Under the assumption that rooms have at most two beds, these fields can hold atomic values. Nevertheless, the approach is less flexible than representing beds in a separate relation. If the business requirements change, we would reconsider this structure rather than continually adding attributes such as bed3_size and bed4_size.

Second Normal Form (2NF)

Second Normal Form requires a relation to satisfy 1NF and have no partial dependencies of non-prime attributes on candidate keys. Our proposed tables use single-attribute primary keys, so partial dependencies on those primary keys cannot arise.

However, we must also check for additional candidate keys and their functional dependencies before concluding that every relation satisfies 2NF.

Third Normal Form (3NF)

Third Normal Form (3NF) is especially important for our database because hotel information is organized across several connected tables. For example, each room belongs to a floor, each floor belongs to a wing, each wing belongs to a building, and each building belongs to a hotel. If we included the hotel's address directly in the Room table, we would end up repeating the same information for every room in that hotel. This could also cause problems if the hotel's address changed, since we would have to update multiple records instead of just one.

To avoid this, we store hotel details in the Hotel table and use foreign keys to connect the related tables. We follow the same approach with customers and reservations. Customer information, such as names and phone numbers, is stored in the Customer table, while details about individual bookings are kept in Reservation. This helps us keep the database organized, reduce unnecessary duplication, and make updates easier.

Based on this structure, we identified the following functional dependencies for each table:

Determinant	**Dependent Attributes**
hotelID	name, address, city, country

buildingID	hotelID, name
wingID	buildingID, wingName
floorID	wingID, floorNumber
roomID	floorID, room_number, number_of_beds, bed1_size, bed2_size, extra_space, smoking, amt_guests
customerID	customerType, name, phone, billingAddress
reservationID	customerID, roomID, reservationTime, checkinTime, checkoutTime, actualRate

These dependencies reflect how we expect the data to be organized, but they don't necessarily mean that our database fully meets the requirements of 3NF yet. Before finalizing the ERD, we still need to check for any additional candidate keys or dependencies we may have overlooked, especially cases where one non-key attribute depends on another. We want to make sure our database is actually normalized, rather than just assuming it is. By reviewing these relationships carefully, we can reduce repeated information and avoid problems when adding, updating, or deleting records in the future.

7. Design Trade-offs and Potential Extensions

One of the decisions we had to make was how much detail to include in our database without making it unnecessarily complicated. In reality, a hotel management system would need to keep track of much more than rooms and reservations. It might include employees, payments, maintenance requests, housekeeping, and other guest services. However, since we are still in the early stages of the project, we decided to focus on the information most relevant to how hotels are organized and how customers make reservations.

While working on the design, we also noticed a few areas that could be improved as the project develops. For example, our current Room table includes separate attributes for the first and second bed. This works if we assume that each room has no more than two beds, but it becomes limiting if a hotel offers rooms with three or more. One possible solution would be to create a separate Bed table connected to Room, allowing us to store different bed configurations without having to add new columns each time.

Another limitation is how we handle reservations. Right now, each reservation is connected to one room. This makes sense for a basic booking system, but what happens if a family wants to reserve three rooms under one booking? Our current design would require separate reservations. In the future, we could introduce an additional table connecting reservations to multiple rooms. We also haven't included a way to track whether a reservation is pending, confirmed, canceled, or completed. This could become important when determining which rooms are actually available. For instance, a canceled reservation shouldn't prevent another customer from booking the same room.

Pricing is another area we could expand. Although we record the actual rate charged for each reservation, our database doesn't currently account for seasonal rates, discounts, or differences in pricing between room categories. These features could be added later if we wanted to make the system more realistic. We don't necessarily see these limitations as problems with our current design. Instead, they helped us think more carefully about what our database needs to accomplish right now and what could be developed further. For this milestone, we wanted to build a structure that makes sense, avoids unnecessary duplication, and can be expanded as we learn more about database design.

8. Conclusion

Working on the Last Resort Hotels database has helped us better understand how different parts of a hotel are connected and why organizing that information properly matters. One of our main decisions was to keep information about the hotels, buildings, and rooms separate from customer and reservation records. This way, we can update room details without affecting past bookings, keep track of returning customers, and easily identify where a particular room is located within a hotel.

For now, we believe our seven-entity design gives us a solid starting point without overcomplicating the database. There are still a few things we need to review, particularly the relationships between tables, functional dependencies, and whether our design fully meets the requirements of Third Normal Form. We also want to make sure that our final ERD matches the assumptions and decisions we've explained throughout this report.

As we continue working on the project, we expect to make adjustments and possibly introduce additional tables as new requirements come up. What matters most to us at this stage is building a database that is logically organized, keeps information consistent, and can support more complex hotel operations in the future.